

Teste Técnico - Desenvolvedor

Parte 1: Teste Teórico

1. Explique o conceito de encapsulamento e como ele pode ser aplicado em uma classe C#. Dê um exemplo prático.
2. Descreva o princípio do Open/Closed e implemente um pequeno exemplo em C# que respeite esse princípio.
3. Qual será a saída do código abaixo? Justifique sua resposta.

```
public class Animal
{
    public virtual void Falar() => Console.WriteLine("Animal faz som");
}

public class Cachorro : Animal
{
    public override void Falar() => Console.WriteLine("Latido");
}

public class Gato : Animal
{
    public new void Falar() => Console.WriteLine("Miau");
}

Animal a1 = new Cachorro();
Animal a2 = new Gato();

a1.Falar();
a2.Falar();
```

4. Qual a diferença entre os modificadores `private`, `protected` e `public` no TypeScript? Dê um exemplo para cada um.
5. O que são pipes no Angular? Como criar um pipe personalizado para formatar um CPF?
6. Considere a tabela Pessoas com os campos (Cpf, Nome, Genero, Endereco, Idade, Municipio, Estado).

Teste Técnico - Desenvolvedor

Escreva uma query SQL para buscar todas as pessoas do estado de SP com idade entre 25 e 40 anos, ordenadas pelo nome.

7. Explique como funciona o ciclo de vida de um componente Angular. Qual método é ideal para fazer uma chamada HTTP quando o componente for exibido?

8. Analise o trecho abaixo:

```
this.http.get('api/pessoas').subscribe(data => {  
  this.pessoas = data;  
});  
  
console.log(this.pessoas);
```

O que está errado ou incompleto nesse código? Como corrigiria?

Parte 2: Desafio Prático

Objetivo: Criar uma aplicação simples Angular 17 integrada com uma API mockada em C#.

Back-end (C# - ASP.NET Core):

- Criar uma WebAPI com endpoint GET /api/pessoas
- Retornar uma lista mockada com 30 registros de pessoas contendo os campos:
 - Cpf (string)
 - Nome (string)
 - Genero (string)
 - Endereco (string)
 - Idade (int)
 - Municipio (string)
 - Estado (string)

Front-end (Angular 17):

Teste Técnico - Desenvolvedor

- Criar uma tela com uma tabela paginada exibindo os registros da API.
- A tabela deve:
 - Exibir 10 registros por página
 - Ter um campo de filtro por Nome
 - Ter um botão para carregar os dados da API

Observações:

- Utilize Angular Material ou outro framework de UI para a tabela e paginação.
- Utilize HttpClient para comunicação com a API.
- A solução deve funcionar localmente, sem necessidade de banco de dados real.
- Envie o projeto com instruções de como executar (README).